JAVASCRIPT – OBJECTS AND JSON

Complete Theory and Practical Examples

PART 1: OBJECTS IN JAVASCRIPT

1. What is an Object?

An object is a collection of related data stored in the form of key-value pairs.

- Key: Property name.
- Value: Data stored inside that property.

Example:

```
let student = {
    name: "Rahul",
    age: 21,
    course: "JavaScript",
    marks: 85
};
```

Here:

Key	Value
name	Rahul
age	21
course	JavaScript
marks	85

Why do we need Objects?

Suppose we want to store student information.

Without an object:

```
let name = "Rahul";
let age = 21;
let course = "JavaScript";
let marks = 85;
```

With an object:

```
let student = {
    name: "Rahul",
    age: 21,
    course: "JavaScript",
    marks: 85
};
```

An object keeps related information together and makes it easier to manage.

---

2. Creating an Object

We generally create objects using curly braces `{}`.

```
let car = {
    brand: "BMW",
    model: "X5",
    price: 8000000
};
```

Syntax:

```
let objectName = {
    key1: value1,
    key2: value2,
    key3: value3
};
```

---

3. Accessing Object Properties

There are two main ways to access properties.

A. Dot Notation

```
let student = {
    name: "Rahul",
    age: 21
};

console.log(student.name);
console.log(student.age);
```

Output:

```
Rahul
21
```

B. Bracket Notation

```
console.log(student["name"]);
console.log(student["age"]);
```

Output:

```
Rahul
21
```

Both methods access the same properties.

When to use Bracket Notation?

Use bracket notation when the property name is stored in a variable.

```
let key = "name";

console.log(student[key]);
```

Output:

```
Rahul
```

---

4. Adding New Properties

We can add properties to an existing object.

```
let student = {
    name: "Rahul",
    age: 21
};

student.course = "JavaScript";
student.marks = 90;
```

```
console.log(student);
```

Updated Object:

```
{
    name: "Rahul",
    age: 21,
    course: "JavaScript",
    marks: 90
}
```

---

5. Updating Properties

We can change an existing property's value.

```
let student = {
    name: "Rahul",
    age: 21
};

student.age = 22;

console.log(student.age);
```

Output:

```
22
```

---

6. Deleting Properties

Use the `delete` keyword to remove a property.

```
let student = {
    name: "Rahul",
    age: 21,
    course: "JavaScript"
};

delete student.age;

console.log(student);
```

Output:

```
{
    name: "Rahul",
    course: "JavaScript"
}
```

---

7. Object Methods

When a function is stored inside an object, it is called a method.

```
let student = {
    name: "Rahul",
    age: 21,

    introduce: function() {
        console.log("Hello, I am " + this.name);
    }
};

student.introduce();
```

Output:

```
Hello, I am Rahul
```

What is `this`?

Inside a regular object method, `this` generally refers to the object that calls the method.

Example:

```
let car = {
    brand: "BMW",

    display: function() {
        console.log(this.brand);
    }
};

car.display();
```

Output:

```
BMW
```

---

8. Nested Objects

An object can contain another object.

```
let student = {
    name: "Rahul",
    age: 21,

    address: {
        city: "Ahmedabad",
        state: "Gujarat"
    }
};

console.log(student.address.city);
console.log(student.address.state);
```

Output:

```
Ahmedabad
Gujarat
```

---

9. Array of Objects

We can store multiple objects inside an array.

This is very useful when working with APIs.

```
let students = [
    {
        name: "Rahul",
        marks: 85
    },
    {
        name: "Priya",
        marks: 92
    },
    {
        name: "Amit",
        marks: 75
    }
];

console.log(students[0].name);
console.log(students[1].marks);
```

Output:

```
Rahul
92
```

Using forEach()

```
students.forEach(function(student) {
    console.log(student.name);
});
```

Output:

```
Rahul
Priya
Amit
```

PART 2: JSON

10. What is JSON?

JSON stands for JavaScript Object Notation.

JSON is a lightweight, text-based format used to store and exchange data between applications.

Example:

```
{
    "name": "Rahul",
    "age": 21,
    "course": "JavaScript",
    "marks": 85
}
```

Why do we need JSON?

Imagine your frontend needs student information from a server.

The server commonly sends data in JSON format.

JavaScript reads this data and converts it into usable values.

JSON is commonly used in:

1. API communication.
2. Sending data from frontend to backend.
3. Receiving data from backend to frontend.
4. Storing structured data.

---

11. JavaScript Object vs JSON

They look similar, but they are different.

JavaScript Object	**JSON**
JavaScript data structure	Text-based data format
Keys can be unquoted identifiers	Keys must use double quotes
Strings can use single or double quotes	Strings must use double quotes
Can contain functions	Cannot contain functions
Can contain undefined	Cannot represent undefined

JavaScript Object

```
let student = {
    name: "Rahul",
    age: 21,

    display: function() {
        console.log(this.name);
```

```
    }
};
```

JSON

```
{
    "name": "Rahul",
    "age": 21
}
```

Notice that JSON does not support functions.

---

12. JSON.stringify()

`JSON.stringify()` converts a JavaScript value into a JSON string.

Conversion:

JavaScript Object → JSON String

Example:

```
let student = {
    name: "Rahul",
    age: 21,
    course: "JavaScript"
};

let data = JSON.stringify(student);

console.log(data);
console.log(typeof data);
```

Output:

```
{"name":"Rahul","age":21,"course":"JavaScript"}
string
```

Why do we need stringify()?

When we want to send JavaScript object data as JSON text or store it as JSON text, we use `JSON.stringify()`.

Example with localStorage:

```
let student = {
    name: "Rahul",
    age: 21
};

let jsonData = JSON.stringify(student);

localStorage.setItem("student", jsonData);
```

---

13. JSON.parse()

`JSON.parse()` converts a valid JSON string into a JavaScript value, usually an object or array.

Conversion:

JSON String → JavaScript Object

Example:

```
let data = '{"name":"Rahul","age":21}';

let student = JSON.parse(data);

console.log(student.name);
console.log(student.age);
console.log(typeof student);
```

Output:

```
Rahul
21
object
```

Why do we need parse()?

When we receive JSON data from an API or retrieve JSON text from storage, we use `JSON.parse()` to convert it into usable JavaScript data.

Example with localStorage:

```
let data = localStorage.getItem("student");

let student = JSON.parse(data);
```

```
console.log(student.name);
```

---

14. Difference Between stringify() and parse()

Method	Conversion	Purpose
JSON.stringify()	Object → JSON String	To send or store data
JSON.parse()	JSON String → JavaScript Value	To use received or stored data

Remember:

```
// Object to JSON String
let jsonData = JSON.stringify(student);

// JSON String to JavaScript Object
let obj = JSON.parse(jsonData);
```

Easy Way to Remember

JavaScript Object | | JSON.stringify() ↓ JSON String | | JSON.parse() ↓ JavaScript Object

---

PART 3: PRACTICAL EXAMPLE

15. Complete Example – Object to JSON and Back

```
let student = {
    id: 101,
    name: "Rahul",
    course: "JavaScript",
    marks: 85
};

// Convert Object into JSON String
let jsonData = JSON.stringify(student);

console.log("JSON Data:");
console.log(jsonData);

// Convert JSON String back into Object
let newStudent = JSON.parse(jsonData);

console.log("Student Name:", newStudent.name);
console.log("Student Marks:", newStudent.marks);
```

Output:

```
JSON Data:
{"id":101,"name":"Rahul","course":"JavaScript","marks":85}

Student Name: Rahul
Student Marks: 85
```

PART 4: JSON IN API COMMUNICATION

16. What is response.json()?

When we call an API using fetch(), the server generally sends a response containing JSON data.

We use `response.json()` to read and parse that response body.

Example:

```
fetch("https://example.com/api/student")
    .then(function(response) {
        return response.json();
    })
    .then(function(data) {
        console.log(data.name);
        console.log(data.course);
    });
```

Important Difference

`response.json()` and `JSON.parse()` are not exactly the same.

JSON.parse()

- Used when we already have a JSON string.
- Converts JSON text into a JavaScript value.

response.json()

- Used with the Fetch API response.
- Reads the HTTP response body.
- Parses its JSON text into a JavaScript value.
- Returns a Promise.

Remember:

```
let data = '{"name":"Rahul"}';

let student = JSON.parse(data);
```

Here we already have a JSON string.

But:

```
fetch("https://example.com/api/student")
    .then(response => response.json())
    .then(data => console.log(data));
```

Here we are reading JSON from an HTTP response.

PART 5: IMPORTANT POINTS TO REMEMBER

1. An object stores data in key-value pairs.
2. Objects are created using `{}`.
3. Use dot notation or bracket notation to access properties.
4. We can add, update and delete object properties.
5. A function inside an object is called a method.
6. `this` generally refers to the calling object inside a regular method.
7. JSON stands for JavaScript Object Notation.
8. JSON is a text-based data exchange format.
9. JSON keys and strings must use double quotes.
10. `JSON.stringify()` converts a JavaScript value into a JSON string.
11. `JSON.parse()` converts a JSON string into a JavaScript value.

12. `response.json()` reads and parses an API response body.

RECOMMENDED TEACHING ORDER

1. Objects Introduction
2. Creating Objects
3. Accessing Properties
4. Adding, Updating and Deleting Properties
5. Object Methods and this
6. Nested Objects
7. Array of Objects
8. JSON Introduction
9. JSON.stringify()
10. JSON.parse()
11. JSON in API Communication
12. response.json()

After completing these topics, students will have the necessary foundation to start working with Fetch API and Axios.